Guía Metodológica

Solución al Ejercicio *HabanaEntregas*

Planificación por Hitos (Sprints) con Arquitectura de 3 Capas

ASP.NET Core MVC · Entity Framework · Patrón Repository

Asignatura: Ingeniería de Software
Tema: Planificación y Gestión de Riesgos
Modalidad: Clase Práctica

Índice

1. Introducción **2**

2. Objetivos de Aprendizaje **2**

3. Arquitectura de la Solución **3**
3.1. Visión general . . . 3
3.2. Estructura de la solución en Visual Studio . . . 3
3.3. Modelo de datos (entidades principales) . . . 3
3.4. Patrón Repository . . . 4

4. Planificación por Hitos (Sprints) **5**
4.1. Visión general de los 4 hitos . . . 5
4.2. Equipo de trabajo y roles . . . 5

5. Hito 1: Configuración inicial, LDAP y CRUD básico **5**
5.1. Objetivo . . . 5
5.2. Tareas detalladas . . . 5
5.3. Ejemplo de configuración LDAP . . . 8
5.4. Criterios de aceptación . . . 8

6. Hito 2: CRUD completo y reportes básicos (Entrega 1) **8**
6.1. Objetivo . . . 9
6.2. Tareas detalladas . . . 9
6.3. Ejemplo de consulta LINQ para el reporte (a) . . . 10
6.4. Criterios de aceptación . . . 10

7. Hito 3: Reportes avanzados y exportación a PDF (Entrega 2) **10**
7.1. Objetivo . . . 10
7.2. Tareas detalladas . . . 10
7.3. Ejemplo de interfaz para exportadores . . . 11
7.4. Criterios de aceptación . . . 11

8. Hito 4: Integración con la intranet y ajustes finales (Entrega 3) **12**
8.1. Objetivo . . . 12
8.2. Tareas detalladas . . . 12
8.3. Ejemplo de endpoint para la intranet . . . 13
8.4. Criterios de aceptación . . . 13

9. Tareas Genéricas para la Entrega Final del Proyecto **13**
9.1. Preparación del Entorno de Producción . . . 13
9.2. Despliegue de la Aplicación . . . 14
9.3. Pruebas de Aceptación (UAT) con el Cliente . . . 14
9.4. Documentación Final del Proyecto . . . 14
9.5. Capacitación a los Usuarios Finales . . . 14
9.6. Plan de Soporte y Mantenimiento . . . 14
9.7. Entrega Formal y Cierre del Proyecto . . . 14
9.8. Preparación para la Transición al Equipo de Mantenimiento . . . 15
9.9. Consideraciones de Licencias y Derechos de Uso . . . 15
9.10. Retiro del Entorno de Desarrollo . . . 15

10. Gestión de Riesgos **15**
10.1. Matriz de riesgos y oportunidades 15
10.2. Estrategias de respuesta 16

11. Herramientas y Tecnologías **16**

12. Diagrama de Gantt (resumen) **17**

13. Entregables **17**

14. Recomendaciones Metodológicas **17**

15. Conclusiones **18**

1 Introducción

La presente guía metodológica tiene como propósito orientar a los equipos de trabajo en la solución del ejercicio **HabanaEntregas**, correspondiente a la Clase Práctica de Planificación y Gestión de Riesgos de la asignatura Ingeniería de Software.

El problema plantea la necesidad de desarrollar una aplicación que permita actualizar el control de medios (equipos electrónicos y mobiliario) de cada sucursal de la empresa HabanaEntregas, con los siguientes requerimientos clave:

- **Autenticación** mediante login/password contra el directorio LDAP corporativo.
- **Dos roles:** administrador (acceso total) y sucursal (gestión de su sucursal).
- **CRUD** de medios electrónicos y mobiliario.
- **Reportes:**
  a) Cantidad de equipos de un mismo tipo desglosado por marcas.
  b) Cantidad total de medios en mal estado desglosados por tipo.
  c) Principales proveedores de equipamiento y mobiliario.
  d) Costo promedio por tipo de equipo y por tipo de mueble.
- **Exportación a PDF** (inicialmente; extensible a otros formatos).
- **Integración con la intranet corporativa** para alimentar el cuadro de mando del director.
- **Interfaz amigable**, con no más de 3 niveles de navegación.
- **Base de datos** SQL Server 2017.

La solución se planifica en **4 hitos (sprints)** sobre una **arquitectura de 3 capas** (presentación, lógica de negocio y acceso a datos) con **ASP.NET Core MVC**, **Entity Framework Core** como ORM y el **patrón Repository**.

2 Objetivos de Aprendizaje

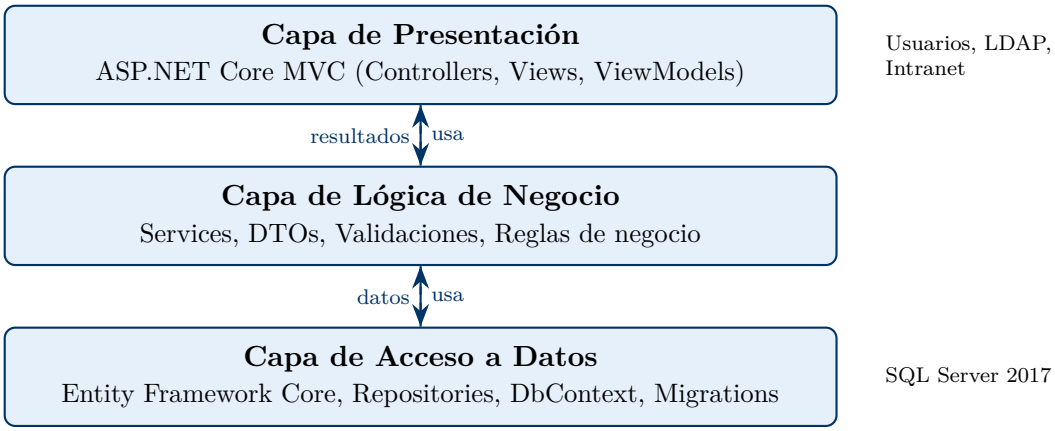
Al finalizar la solución del ejercicio, los estudiantes serán capaces de:

1. **Diseñar** una arquitectura de 3 capas aplicada a un caso real, justificando la separación de responsabilidades.
2. **Implementar** autenticación integrada con LDAP y autorización basada en roles.
3. **Aplicar** el patrón Repository con Entity Framework Core para el acceso a datos.
4. **Desarrollar** funcionalidades CRUD y reportes con exportación a PDF.
5. **Integrar** la aplicación con servicios de la intranet corporativa.
6. **Planificar** el desarrollo en hitos (sprints) con entregables verificables.
7. **Identificar y mitigar** riesgos técnicos y de proyecto.

3 Arquitectura de la Solución

3.1 Visión general

Se adopta una **arquitectura de 3 capas** que promueve la separación de responsabilidades, la mantenibilidad y la testabilidad del sistema.



3.2 Estructura de la solución en Visual Studio

```
HabanaEntregas.sln
|-- HabanaEntregas.Domain          (Entidades, Enums, Interfaces)
|-- HabanaEntregas.Data            (EF Core, DbContext, Repositories, Migrations)
|-- HabanaEntregas.Services        (Logica de negocio, DTOs, Validaciones)
|-- HabanaEntregas.Web             (ASP.NET Core MVC, Controllers, Views)
|-- HabanaEntregas.Tests           (Pruebas unitarias e integracion)
```

Listing 1: Estructura de proyectos de la solucion

3.3 Modelo de datos (entidades principales)

```
// Domain/Entities/Medio.cs (abstracta)
public abstract class Medio
{
    public int Id { get; set; }
    public string NumeroInventario { get; set; }
    public decimal CostoUnitario { get; set; }
    public EstadoMedio Estado { get; set; }   // Bien, Mal, Regular
    public int SucursalId { get; set; }
    public Sucursal Sucursal { get; set; }
}

// Domain/Entities/EquipoElectronico.cs
public class EquipoElectronico : Medio
{
    public TipoEquipo Tipo { get; set; }      // Refrigerador, TV, Ventilador, AC, PC
```

```
16              public Marca Marca { get; set; }          // Sony, Atec
                    Panda, LG
17              public string ProveedorUsual { get; set; }
18              public int AniosGarantia { get; set; }
19          }
20
21          // Domain/Entities/Mobiliario.cs
22          public class Mobiliario : Medio
23          {
24              public TipoMobiliario Tipo { get; set; }  // Silla, Mesa,
                    Buro, MueblePC
25              public string Proveedor { get; set; }
26              public int TiempoEstimadoExplotacion { get; set; }
27              public LocalUbicacion Local { get; set; } // Recepcion,
                    Almacen, Oficina
28          }
29
30          // Domain/Entities/Sucursal.cs
31          public class Sucursal
32          {
33              public int Id { get; set; }
34              public string Nombre { get; set; }
35              public string Direccion { get; set; }
36              public string Telefono { get; set; }
37              public ICollection<Medio> Medios { get; set; }
38          }
```

Listing 2: Entidades del dominio

3.4 Patrón Repository

```
1           // Domain/Interfaces/IRepository.cs
2           public interface IRepository<T> where T : class
3           {
4               Task<T?> GetByIdAsync(int id);
5               Task<IEnumerable<T>> GetAllAsync();
6               Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>>
                    predicate);
7               Task AddAsync(T entity);
8               void Update(T entity);
9               void Remove(T entity);
10              Task<int> SaveChangesAsync();
11          }
12
13          // Domain/Interfaces/IMedioRepository.cs
14          public interface IMedioRepository : IRepository<Medio>
15          {
16              Task<IEnumerable<EquipoElectronico>>
                    GetEquiposPorTipoYMarcaAsync();
17              Task<IEnumerable<Medio>> GetMediosEnMalEstadoAsync();
18              Task<IEnumerable<string>> GetPrincipalesProveedoresAsync();
19              Task<decimal> GetCostoPromedioPorTipoEquipoAsync(TipoEquipo
                    tipo);
20              Task<decimal> GetCostoPromedioPorTipoMobiliarioAsync(
                    TipoMobiliario tipo);
21          }
```

Listing 3: Interfaz generica del repositorio

4 Planificación por Hitos (Sprints)

4.1 Visión general de los 4 hitos

Hito	Objetivo	Entregable principal	Duración
H1	Configuración inicial, autenticación LDAP y CRUD básico de medios	Solución compilable con login LDAP funcional y CRUD de una entidad	2 semanas
H2	CRUD completo (electrónicos y mobiliario) y reportes básicos	**Entrega 1:** CRUD completo + reportes (a) y (b)	2 semanas
H3	Reportes avanzados y exportación a PDF	**Entrega 2:** Reportes (c) y (d) + exportación PDF	2 semanas
H4	Integración con la intranet corporativa y ajustes finales	**Entrega 3:** Integración + sistema final	2 semanas

4.2 Equipo de trabajo y roles

El equipo está compuesto por **5 estudiantes**. Se recomienda la siguiente distribución de roles, rotando responsabilidades en cada hito para garantizar la formación integral:

#	Rol	Responsabilidades principales
1	Líder técnico / Arquitecto	Diseño de arquitectura, revisión de código, integración.
2	Desarrollador Backend	Servicios, lógica de negocio, validaciones.
3	Desarrollador de Datos	EF Core, repositorios, migraciones, consultas.
4	Desarrollador Frontend	Vistas MVC, usabilidad, navegación (máx. 3 niveles).
5	QA / DevOps	Pruebas, CI/CD, documentación, gestión de riesgos.

5 Hito 1: Configuración inicial, LDAP y CRUD básico

5.1 Objetivo

Establecer la base técnica del proyecto, configurar la arquitectura de 3 capas, implementar la autenticación contra LDAP y desarrollar el CRUD básico de una entidad (por ejemplo, `Sucursal`).

5.2 Tareas detalladas

1. **Configurar el proyecto ASP.NET Core MVC con Entity Framework y SQL Server.**

- Crear nuevo proyecto ASP.NET Core MVC.
- Instalar paquetes NuGet: `Microsoft.EntityFrameworkCore.SqlServer`, `Microsoft.EntityFra` `Microsoft.AspNetCore.Identity.EntityFrameworkCore`.
- Configurar cadena de conexión en `appsettings.json`.
- Configurar `DbContext` en `Program.cs`.
- Configurar servicios MVC con controladores y vistas.
- Configurar entorno de desarrollo vs producción.
- Crear estructura de carpetas: Models, Views, Controllers, Data, Services, Repositories.

2. **Configurar la autenticación con LDAP.**
   - Investigar e instalar paquete `Novell.Directory.Ldap.NETStandard`.
   - Crear modelo `LdapConfig` en `appsettings.json`.
   - Implementar servicio `ILdapService`: `bool Authenticate(string username, string password)`, `UserLdap GetUser(string username)`, `List<string>GetUserRoles(string username)`.
   - Crear `CustomAuthenticationHandler`.
   - Configurar políticas de roles: Admin, Sucursal.
   - Implementar claims transformation para roles LDAP.
   - Crear middleware de autenticación personalizada.
3. **Diseñar la base de datos: tablas para Medios, EquiposElectronicos, Mobiliario, Tipos, Marcas, Proveedores, etc.**
   - Diseñar diagrama ER inicial.
   - Crear tablas: Sucursales (Id, Nombre, Direccion, Telefono), Medios (Id, TipoMedio, NumeroInventario, CostoUnitario, Estado, SucursalId), EquiposElectronicos (MedioId, TipoEquipo, Marca, Proveedor, AniosGarantia), Mobiliarios (MedioId, TipoMobiliario, Proveedor, TiempoExplotacion, LocalUbicacion), Usuarios (Id, Username, Nombre, Rol, SucursalId).
   - Definir constraints y relaciones FK.
   - Crear índices para búsquedas frecuentes.
4. **Implementar el patrón Repository para la capa de acceso a datos.**
   - Crear interfaz `IRepository<T>`.
   - Implementar `BaseRepository<T>` con EF Core.
   - Crear interfaces específicas: `IMedioRepository`, `IEquipoElectronicoRepository`, `IMobiliarioRepository`.
   - Implementar repositorios concretos.
   - Crear patrón `UnitOfWork`.
   - Configurar inyección de dependencias en `Program.cs`.
5. **Crear las entidades (models) para Medios, EquiposElectronicos, Mobiliario, etc.**
   - Crear clase base `Medio` (abstracta).
   - Crear clase `EquipoElectronico : Medio`.

- Crear clase `Mobiliario : Medio`.
- Crear enums: `EstadoMedio`, `TipoEquipo`, `TipoMobiliario`, `LocalUbicacion`.
- Configurar herencia TPH en `DbContext`.
- Configurar relaciones y navegación.
- Agregar data annotations para validación.

6. **Implementar el login y logout con LDAP y manejo de roles (Administrador y Sucursal).**
   - Crear `AccountController` con actions: Login (GET/POST), Logout, AccessDenied.
   - Crear `LoginViewModel`.
   - Implementar lógica de autenticación en Login POST.
   - Crear vistas: `Login.cshtml`, `_LoginPartial.cshtml`.
   - Configurar cookies de autenticación.
   - Implementar autorización por roles en controllers.
   - Crear filtros personalizados para autorización.
7. **Crear una página de inicio post-login con un menú básico.**
   - Crear `HomeController` con Index action.
   - Diseñar layout principal `_Layout.cshtml`.
   - Crear menú de navegación responsivo.
   - Implementar `_LoginPartial` para mostrar usuario actual.
   - Crear dashboard con estadísticas básicas.
   - Configurar rutas y navegación.
   - Aplicar estilos CSS/Bootstrap.
8. **CRUD básico para Medios (crear, listar, editar, eliminar) con vistas MVC.**
   - Crear `MediosController` con: Index, Create (GET/POST), Edit (GET/POST), Delete (GET/POST), Details.
   - Crear `MedioViewModel` con validaciones.
   - Implementar vistas Razor para cada acción.
   - Agregar paginación en Index.
   - Implementar búsqueda y filtros.
   - Crear partial views para formularios reutilizables.
9. **Implementar la base de datos con migraciones de Entity Framework.**
   - Crear migración inicial: `Add-Migration InitialCreate`.
   - Actualizar base de datos: `Update-Database`.
   - Crear datos semilla (seed data) para testing.
   - Configurar estrategia de migraciones en producción.
   - Crear scripts de migración SQL.
   - Implementar `DbInitializer` para datos iniciales.

5.3 Ejemplo de configuración LDAP

```
        // Services/LdapAuthenticationService.cs
        public class LdapAuthenticationService : IAuthenticationService
        {
            private readonly LdapSettings _settings;

            public LdapAuthenticationService(IOptions<LdapSettings>
                options)
            {
                _settings = options.Value;
            }

            public async Task<UsuarioLdap?> AuthenticateAsync(string
                username, string password)
            {
                using var connection = new LdapConnection(new
                    LdapDirectoryIdentifier(_settings.Host, _settings.
                    Port));
                connection.AuthType = AuthType.Basic;
                connection.Credential = new NetworkCredential(
                $"{username}@{_settings.Domain}", password);

                try
                {
                    await Task.Run(() => connection.Bind());
                    // Recuperar atributos del usuario (rol, sucursal,
                        etc.)
                    return new UsuarioLdap { Username = username };
                }
                catch (LdapException)
                {
                    return null;
                }
            }
        }
```

Listing 4: Servicio de autenticacion LDAP

5.4 Criterios de aceptación

- La solución compila y ejecuta sin errores.
- Un usuario válido de LDAP puede iniciar sesión.
- Un usuario inválido es rechazado.
- El rol Administrador puede acceder al CRUD de Sucursal.
- El rol Sucursal solo ve su propia sucursal.
- La navegación no supera los 3 niveles.

6 Hito 2: CRUD completo y reportes básicos (Entrega 1)

6.1 Objetivo

Completar el CRUD de medios electrónicos y mobiliario, e implementar los reportes (a) y (b).

6.2 Tareas detalladas

1. **Completar el CRUD para EquiposElectrónicos y Mobiliario (incluyendo las relaciones con Tipo, Marca, Proveedor, etc.).**
   - Crear `EquiposElectronicosController` con CRUD completo.
   - Crear `MobiliariosController` con CRUD completo.
   - Implementar ViewModels específicos para cada tipo.
   - Crear vistas específicas para Create/Edit de cada tipo.
   - Implementar lógica de herencia en repositorios.
   - Agregar validaciones específicas por tipo.
   - Implementar dropdowns para enums y datos maestros.
2. **Implementar vistas específicas para EquiposElectrónicos y Mobiliario.**
   - Crear vistas Index para EquiposElectronicos.
   - Crear vistas Index para Mobiliarios.
   - Diseñar formularios específicos para cada tipo.
   - Implementar selección de tipo con JavaScript dinámico.
   - Crear partial views para campos comunes.
   - Aplicar estilos diferenciados por tipo.
3. **Implementar la lógica de negocio para los reportes (a) y (b).**
   - Crear `ReporteService` con métodos: `GetEquiposPorTipoYMarca()`, `GetMediosEnMalEstadoPor`
   - Implementar consultas LINQ complejas.
   - Crear DTOs para reportes: `EquiposTipoMarcaDTO`, `MediosMalEstadoDTO`.
   - Optimizar consultas con EF Core.
   - Agregar caching para datos frecuentes.
4. **Crear vistas para los reportes (a) y (b).**
   - Crear `ReportesController` con actions: `EquiposPorTipoMarca()`, `MediosMalEstado()`.
   - Diseñar vistas de reportes con tablas y gráficos.
   - Implementar Chart.js para gráficos básicos.
   - Agregar filtros por fecha y sucursal.
   - Crear partial views para componentes de reportes.
5. **Mejorar la interfaz de usuario para que sea amigable y no tener más de 3 niveles de navegación.**
   - Revisar y optimizar navegación (máximo 3 niveles).
   - Mejorar responsive design con Bootstrap.
   - Agregar iconos y tooltips.
   - Implementar datatables para listas grandes.
   - Agregar confirmaciones para acciones destructivas.
   - Mejorar mensajes de feedback al usuario.

6.3 Ejemplo de consulta LINQ para el reporte (a)

```
        public async Task<IEnumerable<EquipoPorMarcaDto>>
            GetEquiposPorTipoYMarcaAsync()
        {
            return await _context.EquiposElectronicos
            .GroupBy(e => new { e.Tipo, e.Marca })
            .Select(g => new EquipoPorMarcaDto
            {
                Tipo = g.Key.Tipo.ToString(),
                Marca = g.Key.Marca.ToString(),
                Cantidad = g.Count()
            })
            .OrderBy(x => x.Tipo).ThenBy(x => x.Marca)
            .ToListAsync();
        }
```

Listing 5: Reporte de equipos por tipo y marca

6.4 Criterios de aceptación

- CRUD completo y funcional para ambos tipos de medios.
- Reportes (a) y (b) visibles en la interfaz con datos reales.
- Validaciones activas y mensajes de error claros.
- Pruebas unitarias con cobertura mayor o igual al 60 % en servicios y repositorios.

7 Hito 3: Reportes avanzados y exportación a PDF (Entrega 2)

7.1 Objetivo

Implementar los reportes (c) y (d), y habilitar la exportación a PDF de todos los reportes.

7.2 Tareas detalladas

1. **Implementar los reportes restantes (c) y (d).**
   - Extender `ReporteService` con: `GetProveedoresPrincipales()`, `GetCostosPromedioPorTipo()`
   - Crear DTOs: `ProveedorPrincipalDTO`, `CostoPromedioDTO`.
   - Implementar lógica de cálculo para “proveedores principales”.
   - Crear consultas agregadas para promedios.
   - Optimizar consultas con índices apropiados.
2. **Implementar la exportación a PDF para todos los reportes.**
   - Instalar paquete `iTextSharp` o `QuestPDF`.
   - Crear `PdfExportService` con métodos: `ExportEquiposPorTipoMarca()`, `ExportMediosMalEsta` `ExportProveedoresPrincipales()`, `ExportCostosPromedio()`.
   - Diseñar templates PDF para cada reporte.
   - Implementar estilos CSS para PDF.
   - Agregar botones de exportación en vistas.

- Manejar generación asíncrona de PDFs.

3. **Crear vistas para los reportes (c) y (d).**
   - Extender `ReportesController` con: `ProveedoresPrincipales()`, `CostosPromedio()`.
   - Crear vistas Razor para nuevos reportes.
   - Implementar gráficos comparativos.
   - Agregar opciones de filtrado avanzado.
   - Crear vistas parciales para componentes reutilizables.
4. **Realizar pruebas de los reportes y la exportación a PDF.**
   - Crear tests unitarios para `ReporteService`.
   - Probar generación de PDF con datos reales.
   - Validar cálculos de reportes con datos de prueba.
   - Probar performance con grandes volúmenes de datos.
   - Realizar pruebas de integración de exportación.

7.3 Ejemplo de interfaz para exportadores

```
        public interface IReportExporter
        {
            string Format { get; }        // "pdf", "xlsx", "csv"
            byte[] Export<T>(IEnumerable<T> data, string title);
        }

        public class PdfReportExporter : IReportExporter
        {
            public string Format => "pdf";

            public byte[] Export<T>(IEnumerable<T> data, string title)
            {
                // Implementacion con QuestPDF
                var document = Document.Create(container =>
                {
                    container.Page(page =>
                    {
                        page.Header().Text(title).FontSize(18).Bold();
                        page.Content().Table(table =>
                        {
                            // Construir tabla a partir de data
                        });
                    });
                });
                return document.GeneratePdf();
            }
        }
```

Listing 6: Interfaz de exportacion extensible

7.4 Criterios de aceptación

- Reportes (c) y (d) funcionales y visibles.
- Exportación a PDF operativa para los 4 reportes.

- Arquitectura preparada para añadir nuevos formatos sin refactor mayor.
- PDFs con formato profesional (encabezado, fecha, tabla, totales).

8 Hito 4: Integración con la intranet y ajustes finales (Entrega 3)

8.1 Objetivo

Integrar la aplicación con la intranet corporativa para alimentar el cuadro de mando del director, y realizar los ajustes finales.

8.2 Tareas detalladas

1. **Investigar y definir el formato de datos requerido por la intranet corporativa para el cuadro de mando.**
   - Reunión con equipo de intranet para especificaciones.
   - Documentar formato JSON/XML requerido.
   - Identificar bibliotecas de gráficos usadas en intranet.
   - Crear prototipo de estructura de datos compatible.
   - Definir contrato de API con intranet.
2. **Implementar un servicio (API o método) que exponga los datos necesarios para el cuadro de mando en el formato requerido.**
   - Crear `IntranetService` con métodos: `PrepareDashboardData()`, `SendToIntranet()`.
   - Crear DTOs compatibles con intranet: `DashboardDataDTO`, `ChartDataDTO`.
   - Implementar API Controller con endpoints: `GET /api/dashboard/data`, `POST /api/dashboard/update`.
   - Configurar autenticación API.
   - Implementar formatos alternativos (JSON/XML).
3. **Realizar pruebas de integración con la intranet (simulada o real) para asegurar la compatibilidad.**
   - Configurar entorno de pruebas de integración.
   - Crear cliente HTTP para tests de intranet.
   - Implementar tests end-to-end.
   - Validar visualización de datos en cuadro de mando.
   - Probar escenarios de error y recuperación.
   - Realizar pruebas de carga y rendimiento.
4. **Ajustes finales de la aplicación basados en feedback del cliente.**
   - Recopilar feedback del cliente.
   - Priorizar y planificar ajustes.
   - Implementar mejoras de UI/UX.
   - Optimizar performance.
   - Corregir bugs reportados.

- Realizar pruebas de aceptación con cliente.

5. **Documentación y preparación para el despliegue.**
   - Documentar arquitectura y decisiones técnicas.
   - Crear manual de usuario.
   - Documentar procedimientos de instalación.
   - Preparar scripts de despliegue.
   - Configurar IIS y SQL Server en producción.
   - Realizar deploy final y smoke tests.
   - Entregar documentación al cliente.

8.3 Ejemplo de endpoint para la intranet

```
[ApiController]
[Route("api/intranet/[controller]")]
[Authorize(Roles = "Administrador")]
public class DashboardController : ControllerBase
{
    private readonly IMedioService _medioService;

    public DashboardController(IMedioService medioService)
    {
        _medioService = medioService;
    }

    [HttpGet("resumen")]
    public async Task<IActionResult> GetResumen()
    {
        var resumen = await _medioService.
            GetResumenParaDashboardAsync();
        return Ok(resumen);   // JSON compatible con la intranet
    }
}
```

Listing 7: Controlador API para la intranet

8.4 Criterios de aceptación

- La intranet recibe y visualiza correctamente los datos enviados.
- Compatibilidad verificada con las bibliotecas de gráficos de la intranet.
- Seguridad revisada y sin vulnerabilidades críticas.
- Documentación completa (técnica y de usuario).
- Sistema desplegado en IIS con SQL Server 2017.

9 Tareas Genéricas para la Entrega Final del Proyecto

9.1 Preparación del Entorno de Producción

- Configurar el servidor de producción (Windows Server con IIS) con las características necesarias.

- Instalar y configurar SQL Server 2017 en el servidor de producción.
- Configurar la base de datos en producción: ejecutar scripts de migración y datos iniciales.
- Configurar la autenticación LDAP en el entorno de producción.

9.2 Despliegue de la Aplicación

- Publicar la aplicación en el servidor de producción (IIS).
- Configurar los ajustes de la aplicación (`appsettings.json`) para el entorno de producción.
- Realizar pruebas de humo en el entorno de producción para verificar que la aplicación funciona correctamente.

9.3 Pruebas de Aceptación (UAT) con el Cliente

- Coordinar con el cliente una sesión de pruebas de aceptación.
- Documentar y corregir los issues encontrados durante la UAT.

9.4 Documentación Final del Proyecto

- Documentación técnica: arquitectura, manual de instalación, manual de administración.
- Documentación de usuario: manual de usuario, guías de uso de los módulos.
- Documentación de la base de datos: diagrama ER, descripción de tablas y procedimientos.

9.5 Capacitación a los Usuarios Finales

- Preparar material de capacitación (presentaciones, guías, videos).
- Realizar sesiones de capacitación para los roles de administrador y sucursal.
- Realizar una capacitación específica para el administrador del sistema (mantenimiento, backups, etc.).

9.6 Plan de Soporte y Mantenimiento

- Establecer un plan de soporte post-implementación (por ejemplo, 3 meses de soporte inicial).
- Documentar los procedimientos de backup y recuperación.
- Configurar monitoreo de la aplicación (rendimiento, errores, etc.).

9.7 Entrega Formal y Cierre del Proyecto

- Realizar la entrega formal de la aplicación y documentación al cliente.
- Obtener la aceptación formal del cliente (acta de entrega y aceptación).
- Realizar una retrospectiva del proyecto con el equipo.

9.8 Preparación para la Transición al Equipo de Mantenimiento

- Si aplica, capacitar al equipo de mantenimiento del cliente.
- Entregar toda la documentación y código fuente.
- Establecer los canales de comunicación para soporte.

9.9 Consideraciones de Licencias y Derechos de Uso

- Verificar que se cuentan con las licencias necesarias para el software utilizado (SQL Server, librerías de terceros, etc.).

9.10 Retiro del Entorno de Desarrollo

- Archivar el código fuente y la documentación en un repositorio seguro.
- Desmantelar los entornos de desarrollo y prueba si ya no se necesitan.

10 Gestión de Riesgos

10.1 Matriz de riesgos y oportunidades

ID	Riesgo / Oportunidad	Prob.	Impacto	Plan de respuesta
R1	Cambios en los requisitos (agregar medios de transporte)	Media	Alto	Mitigar: backlog priorizado, sprints cortos, validación con cliente al final de cada hito.
R2	Expectativas no cubiertas de los involucrados	Media	Alto	Mitigar: reuniones quincenales, demos funcionales, acta de acuerdos.
R3	Límites del proyecto poco definidos	Alta	Alto	Mitigar: definir alcance por hito, documento de alcance firmado.
R4	Incompatibilidad con bibliotecas de la intranet	Media	Alto	Mitigar: prototipo temprano de integración en Hito 1, validar formato JSON.
R5	Dependencia del directorio LDAP	Baja	Alto	Mitigar: capa de abstracción `IAuthenticationService`, mock para pruebas.
R6	Curva de aprendizaje de ASP.NET Core / EF Core	Media	Medio	Mitigar: pair programming, sesiones de estudio, documentación interna.
R7	Pérdida de código o datos	Baja	Alto	Mitigar: repositorio Git con ramas, commits frecuentes, backups de BD.

ID	Riesgo / Oportunidad	Prob.	Impacto	Plan de respuesta
R8	Baja usabilidad de la interfaz	Media	Medio	Mitigar: pruebas con usuarios, revisión de navegación (máx. 3 niveles).
O1	Clientes disponibles y colaborativos	Alta	Alto	Explotar: involucrarlos en demos y validaciones, obtener retroalimentación temprana.
O2	Licencia SQL Server 2017 e IIS disponibles	Alta	Medio	Explotar: desplegar en entorno real desde Hito 2, pruebas de carga.
O3	Conocimiento del cliente en TI	Alta	Medio	Explotar: delegar validaciones técnicas, co-crear prototipos.
O4	Equipo de 5 estudiantes con roles definidos	Alta	Alto	Explotar: rotación de roles, especialización progresiva, mentoría interna.

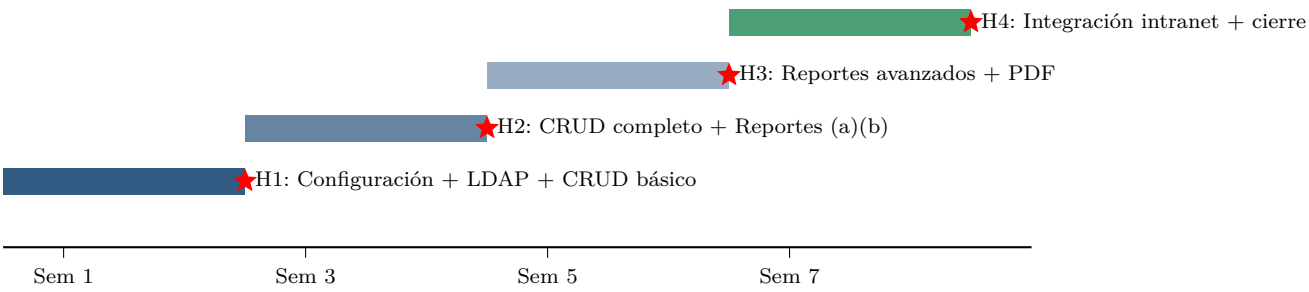
10.2 Estrategias de respuesta

- **Mitigar:** reducir probabilidad o impacto (R1–R8).
- **Explotar:** aprovechar la oportunidad para beneficio del proyecto (O1–O4).
- **Aceptar:** para riesgos residuales de bajo impacto.
- **Transferir:** para riesgos externos (por ejemplo, soporte de LDAP).

11 Herramientas y Tecnologías

Categoría	Tecnología / Herramienta
Framework web	ASP.NET Core MVC (.NET 8)
ORM	Entity Framework Core
Base de datos	SQL Server 2017
Autenticación	LDAP (Novell.Directory.Ldap.NETStandard)
Repositorios	Patrón Repository + Unit of Work
Reportes PDF	QuestPDF / iTextSharp
Frontend	Razor Views, Bootstrap 5, Chart.js
Pruebas	xUnit, Moq
Control de versiones	Git + GitHub / Azure DevOps
Gestión de proyecto	Trello / Azure Boards (Kanban)
CI/CD	GitHub Actions / Azure Pipelines
Servidor	IIS sobre Windows Server

12 Diagrama de Gantt (resumen)



Nota: cada hito tiene una duración estimada de 2 semanas. Las estrellas rojas representan los puntos de entrega (H2, H3 y H4 son las entregas 1, 2 y 3 respectivamente).

13 Entregables

Entrega	Hito	Contenido	Semana
—	H1	Configuración, LDAP, CRUD básico	2
E1	H2	CRUD completo + reportes (a) y (b)	4
E2	H3	Reportes (c) y (d) + exportación PDF	6
E3	H4	Integración con intranet + sistema final	8

14 Recomendaciones Metodológicas

1. **Trabajo colaborativo:** usar Git con ramas por funcionalidad (`feature/`, `hotfix/`) y revisiones de código entre pares.
2. **Reuniones diarias:** 15 minutos para sincronizar avances y bloqueos.
3. **Demos al cliente:** al final de cada hito, mostrar funcionalidad real y recoger retroalimentación.
4. **Documentación continua:** mantener README, wiki y diagramas actualizados en cada hito.
5. **Pruebas desde el inicio:** no dejar las pruebas para el final; escribir pruebas unitarias junto con el código.
6. **Gestión de riesgos viva:** revisar la matriz de riesgos al inicio de cada hito y actualizarla según cambie el contexto.
7. **Enfoque en usabilidad:** validar la navegación (máx. 3 niveles) con usuarios reales o simulados.
8. **Seguridad por diseño:** aplicar principios de OWASP Top 10 desde el Hito 1.

15 Conclusiones

La guía presentada permite a los equipos de 5 estudiantes abordar el ejercicio **HabanaEntregas** de forma estructurada, aplicando buenas prácticas de ingeniería de software:

- La **arquitectura de 3 capas** con ASP.NET Core MVC, EF Core y el patrón Repository facilita la separación de responsabilidades, la testabilidad y el mantenimiento.
- La **planificación en 4 hitos** permite entregas incrementales y funcionales, alineadas con las expectativas del cliente.
- La **gestión de riesgos** identifica los principales peligros (cambios de requisitos, límites difusos, incompatibilidad con la intranet) y define respuestas concretas.
- El **equipo de 5 estudiantes** con roles definidos y rotativos garantiza cobertura de todas las áreas del proyecto.
- Las **tareas genéricas de entrega final** aseguran una transición ordenada a producción, con documentación, capacitación y plan de soporte.